

Schema-Governed Messaging

Apicurio Registry as the schema source-of-truth for [Protobuf](#) and [JSON Schema](#) — enforced as a [CI merge gate](#) with GitHub Actions

A working POC: Apicurio Registry 3.2.0 · RabbitMQ · Spring Boot 4.0 on Java 25

The problem we're solving

A **producer** publishes `OrderCreated` ; a **consumer** reads it. They agree on the shape *today*.

Then someone adds a field, renames one, or changes a type. With no enforced contract:

- The consumer **silently misreads** the message — or crashes in production.
- Coordination by Slack/docs/tribal knowledge **does not scale** and **fails quietly**.
- The breakage is discovered *at runtime*, in the consumer, often hours after the deploy.

The ask: move the breaking-change detection **left** — to the pull request — and make a central registry the **single source of truth** for every message contract.

What a schema registry is

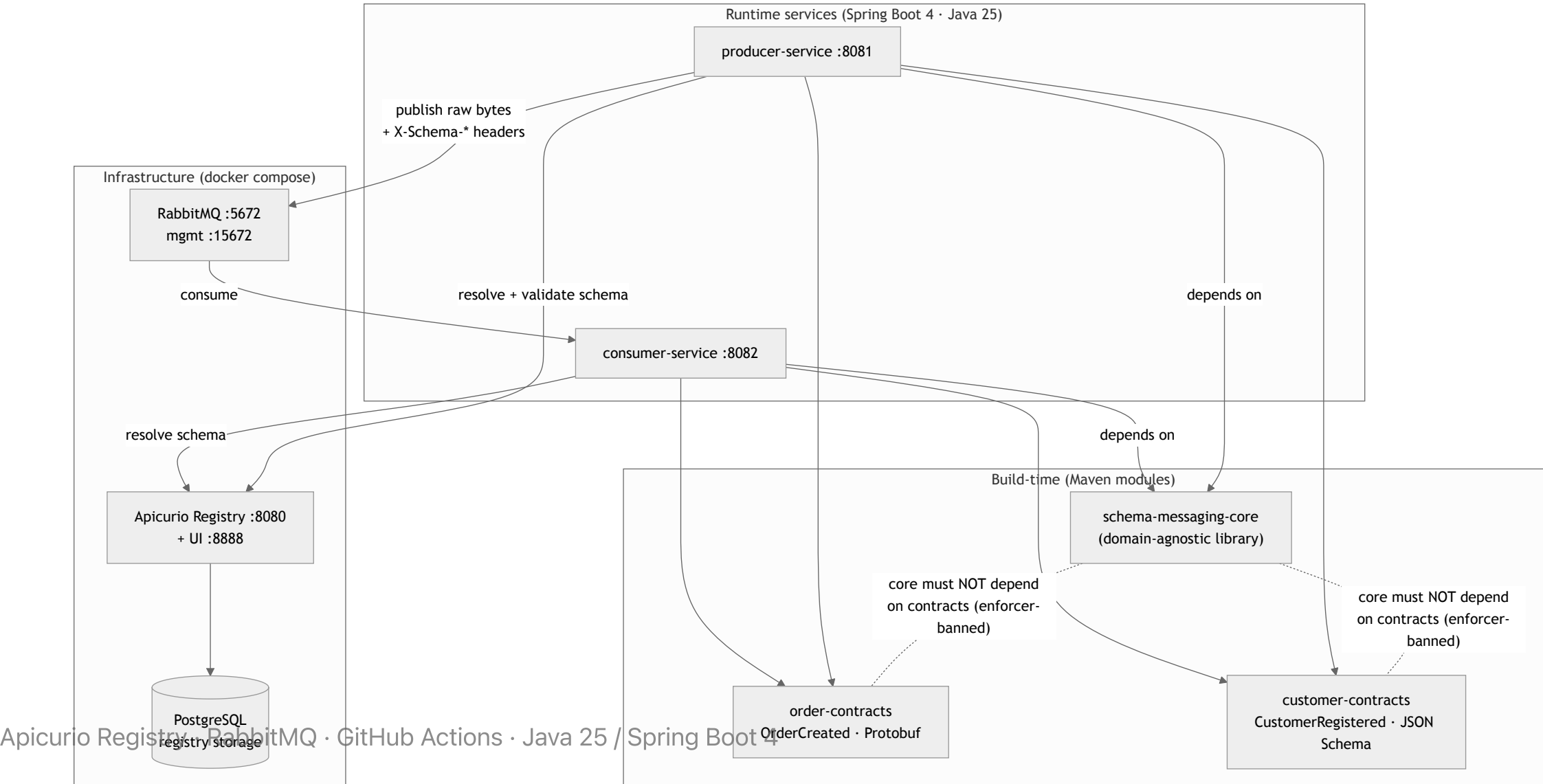
Apicurio Registry is a central service that stores the official, **versioned** definition of each message type, addressed as `group / artifact-id`, with a **compatibility rule** attached.

Term	Meaning
Artifact	A named schema (e.g. <code>OrderCreated</code>)
Version	One immutable snapshot of an artifact (v1, v2, ...)
Compatibility rule	Policy attached to an artifact: <i>which changes are allowed?</i>
Group	A namespace for organizing artifacts (like a folder)

This POC governs **two artifacts of two different schema types** — deliberately, to prove the registry handles both:

Group	Artifact	Schema type	Rule
<code>events.orders</code>	<code>OrderCreated</code>	Protobuf	BACKWARD
<code>events.customers</code>	<code>CustomerRegistered</code>	JSON Schema	FORWARD

System architecture



Two schema types, two strategies

The center of the design is `SchemaAwareMessageConverter` (a Spring AMQP `MessageConverter`).
Format-specific behavior lives behind a **`SerializationStrategy` SPI**:

	Protobuf	JSON Schema
Strategy	<code>ProtobufStrategy</code>	<code>JsonSchemaStrategy</code>
Contracts module	<code>order-contracts</code>	<code>customer-contracts</code>
Depends only on	<code>protobuf-java</code>	Jackson
Content-type	<code>application/x-protobuf</code>	<code>application/json</code>

Each contracts module contributes **one `TypeMapping` bean**: *Java type ↔ registry coordinates ↔ schema type ↔ routing key*. Core stays domain-agnostic; adding a third format = adding a strategy

- a mapping, no converter changes.

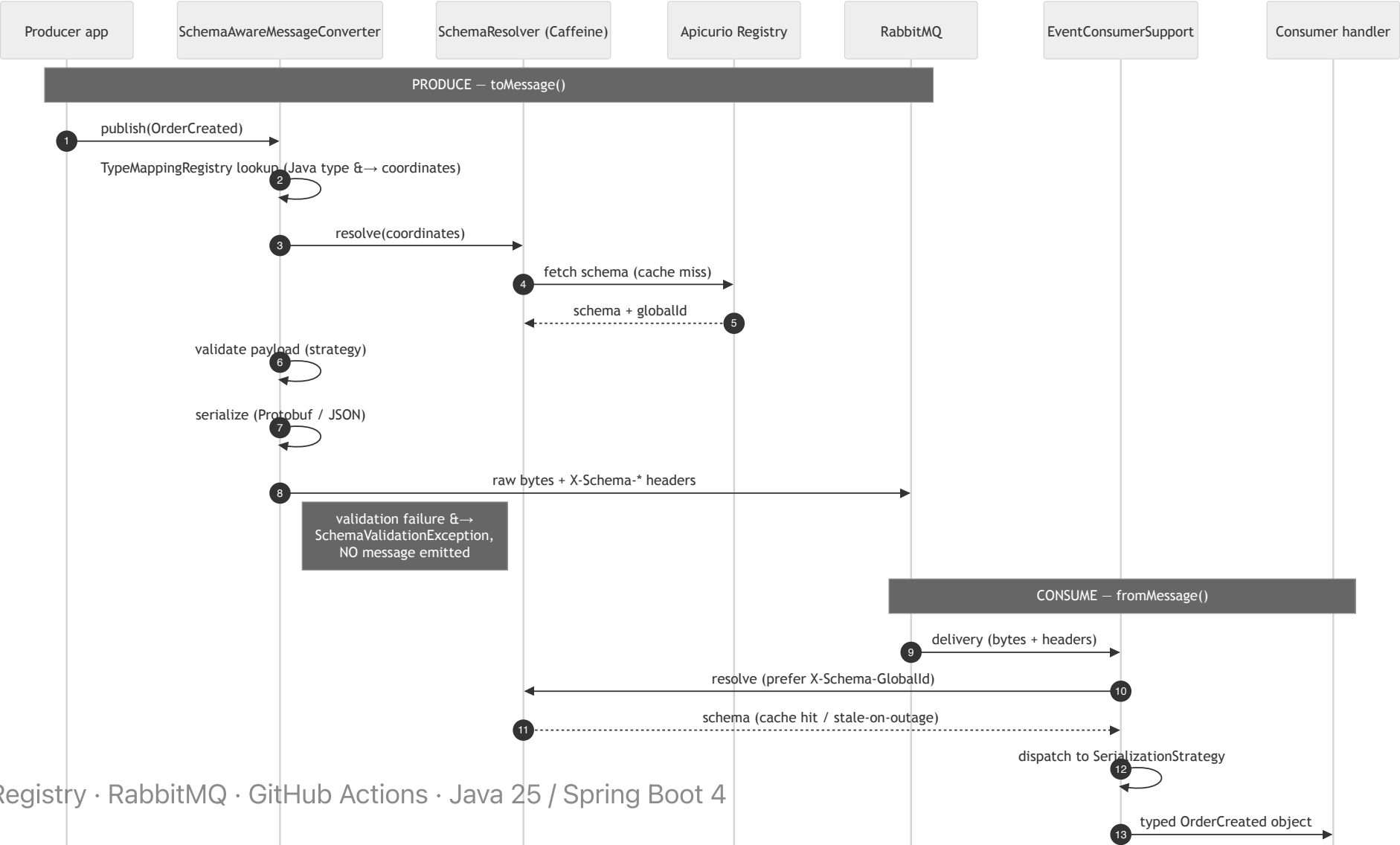
The wire format (strict)

The message body is the **raw serialized bytes only** — pure Protobuf (no magic byte, no length prefix) or plain JSON. **Schema identity travels entirely in headers.**

```
AMQP message  
headers:  
  X-Schema-GlobalId    = 42          ← fast path for consumer  
  X-Schema-Group       = events.orders  
  X-Schema-Artifact    = OrderCreated  
  X-Schema-Version     = 2  
  X-Message-Id / X-Correlation-Id  
content-type: application/x-protobuf | application/json  
body: <raw serialized bytes – nothing else>
```

No vendor framing in the payload → any consumer (even non-Java) can read the bytes given the headers. The registry is the only place schema *content* lives.

A message's life: produce & consume



Schema resolution & caching

`SchemaResolver` sits between the converter and the registry, backed by a **Caffeine cache** (`byGlobalId` + `byCoordinates`):

- **TTL + refresh-after-write** — hot schemas refresh in the background; reads stay fast.
- **Pre-warms on startup** — known coordinates resolved before the first message flows.
- **Serves stale on registry outage** — if Apicurio is down, cached schemas keep traffic moving.
- Throws `RegistryUnavailableException` **only when nothing is cached** — fail, but not eagerly.

Design point: the registry is on the **control plane**, not the per-message hot path. A registry blip degrades to "serve from cache", not "stop processing".

Compatibility governance

The core of "schema-governed"

Same registry, two formats, two rules — and *why*

BACKWARD — the Protobuf story

Rule: BACKWARD = *new code must read old data (new code, old data → works).*

Why Protobuf fits BACKWARD: Protobuf serializes by **field number**, not name. An unknown field number is silently skipped.

```
// v1 (fields 1-7)
message OrderCreated {
  string order_id    = 1;
  ...
  string created_at  = 7;
}

// v2 - current (adds 8-10), BACKWARD-compatible
message OrderCreated {
  string order_id    = 1;
  ... // 2-7 unchanged
  optional string promo_code = 8;
  optional string notes      = 9;
  optional string input1     = 10;
}
```

✓ **Add an optional field with a new number** — old consumers ignore 8-10.

✗ **Change a field's type / reuse a number:**

```
int64 order_id = 1; // WAS string - same number, different wire type → garbled decode → REJECTED
```

FORWARD — the JSON Schema story

Rule: `FORWARD` = *old code must read new data (old code, new data → works).*

The subtlety that trips everyone up: in Apicurio's JSON Schema checker, adding **any** property — even an optional one — is classified as `OBJECT_TYPE_PROPERTY_SCHEMAS_NARROWED`.

- **BACKWARD rejects** adding an optional property (sees it as "narrowing").
- **FORWARD accepts** it (old consumers using Jackson ignore unknown fields by default).

```
// v3 – current: promoCode, input1 added as OPTIONAL → FORWARD-compatible
"properties": { ...,
  "promoCode": { "type": "string" }, // added v2
  "input1":     { "type": "string" } }, // added v3
"required": ["customerId","email","firstName","lastName"] // unchanged
```

✗ FORWARD-incompatible: add `accountType` to `required` — old code can't satisfy a requirement it never knew about.

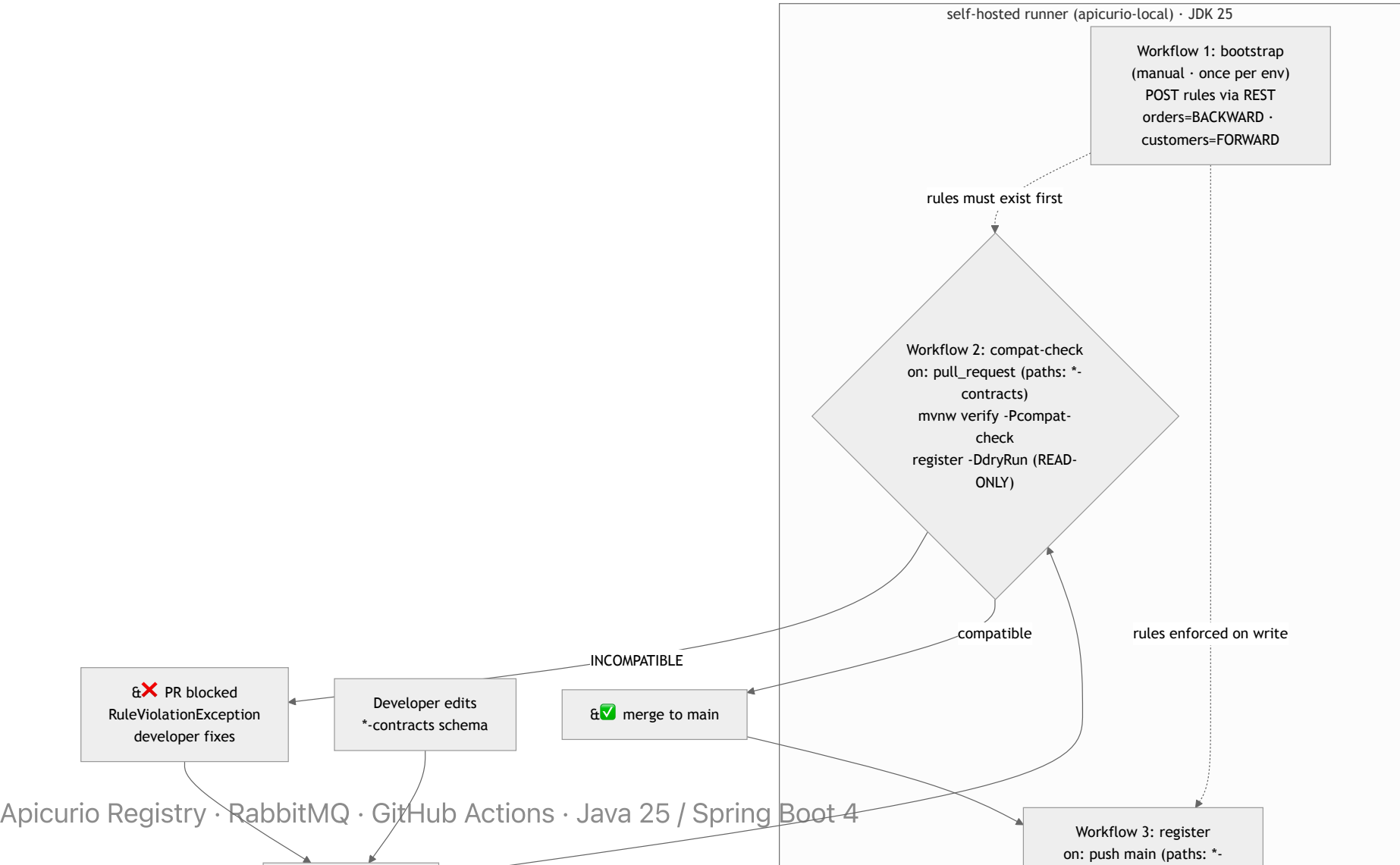
Compatibility modes at a glance

Mode	Apicurio checks	Typical use
BACKWARD	New schema reads old data	Protobuf / Avro — consumers upgraded first
FORWARD	Old schema reads new data	JSON Schema — consumers may lag producers
FULL	Both directions hold	High-confidence environments
NONE	No checking	Dev / prototyping only
*_TRANSITIVE	Same, but vs every prior version	Long-lived schemas, many lagging consumers

This POC: OrderCreated → BACKWARD · CustomerRegistered → FORWARD.

The rule is a **per-artifact policy** — you choose it to match how that format evolves and how your producers/consumers are deployed relative to each other.

Governance via CI — three workflows



Workflow 1 — bootstrap (once per environment)

`schema-governance-bootstrap.yml` · trigger: **manual** (`workflow_dispatch`)

Attaches the compatibility rules via the Apicurio REST API. **Idempotent**: a 409 (rule already exists) is treated as success.

```
# Attach BACKWARD to the Protobuf artifact
curl -X POST ".../v3/groups/events.orders/artifacts/OrderCreated/rules" \
  -H 'Content-Type: application/json' \
  -d '{"ruleType":"COMPATIBILITY","config":"BACKWARD"}'

# Attach FORWARD to the JSON Schema artifact
curl -X POST ".../v3/groups/events.customers/artifacts/CustomerRegistered/rules" \
  -H 'Content-Type: application/json' \
  -d '{"ruleType":"COMPATIBILITY","config":"FORWARD"}'
```

Without this step, the other two workflows have no rules to enforce — schemas could change in any way and Apicurio would accept them silently.

Workflow 2 — compat-check (the merge gate)

`schema-compat-check.yml` · trigger: **every PR** touching `order-contracts/**` or `customer-contracts/**`

```
on:
  pull_request:
    paths: ['order-contracts/**', 'customer-contracts/**']
  steps:
    - run: |
        ./mvnw -pl order-contracts,customer-contracts verify -Pcompat-check \
          -Dapicurio.registry.url=http://localhost:8080
```

The `-Pcompat-check` profile runs the official `apicurio-registry-maven-plugin` with `dryRun=true` — it validates the PR's schema against the **real registered history** but writes nothing.

- **PASS** → `BUILD SUCCESS`, PR may proceed.
- **FAIL** → `BUILD FAILURE` + `RuleViolationException`, **merge blocked**.

Workflow 3 — register (post-merge write)

`schema-register.yml` · trigger: **push to main** touching the contracts modules

```
on:
  push:
    branches: [main]
    paths: ['order-contracts/**', 'customer-contracts/**']
  steps:
    - run: |
        ./mvnw -pl order-contracts,customer-contracts apicurio-registry:register \
        -Dapicurio.registry.url=http://localhost:8080
```

- Configured with `ifExists=FIND_OR_CREATE_VERSION` → **idempotent**; re-runs never create dupes.
- Runs **only on main** — by the time it runs, Workflow 2 already proved compatibility, so the write should always succeed.

Net effect: the registry's contents are a **projection of main**. What's merged is what's registered — no manual registration drift.

The gate in action — decision matrix

Format	Change	Rule	Outcome
Protobuf	Add optional field, new number	BACKWARD	✓ PASS
Protobuf	Change a field's type / reuse a number	BACKWARD	✗ BLOCKED
JSON Schema	Add an optional property	FORWARD	✓ PASS
JSON Schema	Add a required property	FORWARD	✗ BLOCKED

A blocked PR shows exactly this:

```
[ERROR] PUT .../artifacts/OrderCreated/versions?dryRun=true → HTTP 409
[ERROR] RuleViolationException: Incompatible schema change detected.
[INFO] BUILD FAILURE
```

The developer sees the failure **on the PR**, with the offending change named — not a 2 a.m. page.

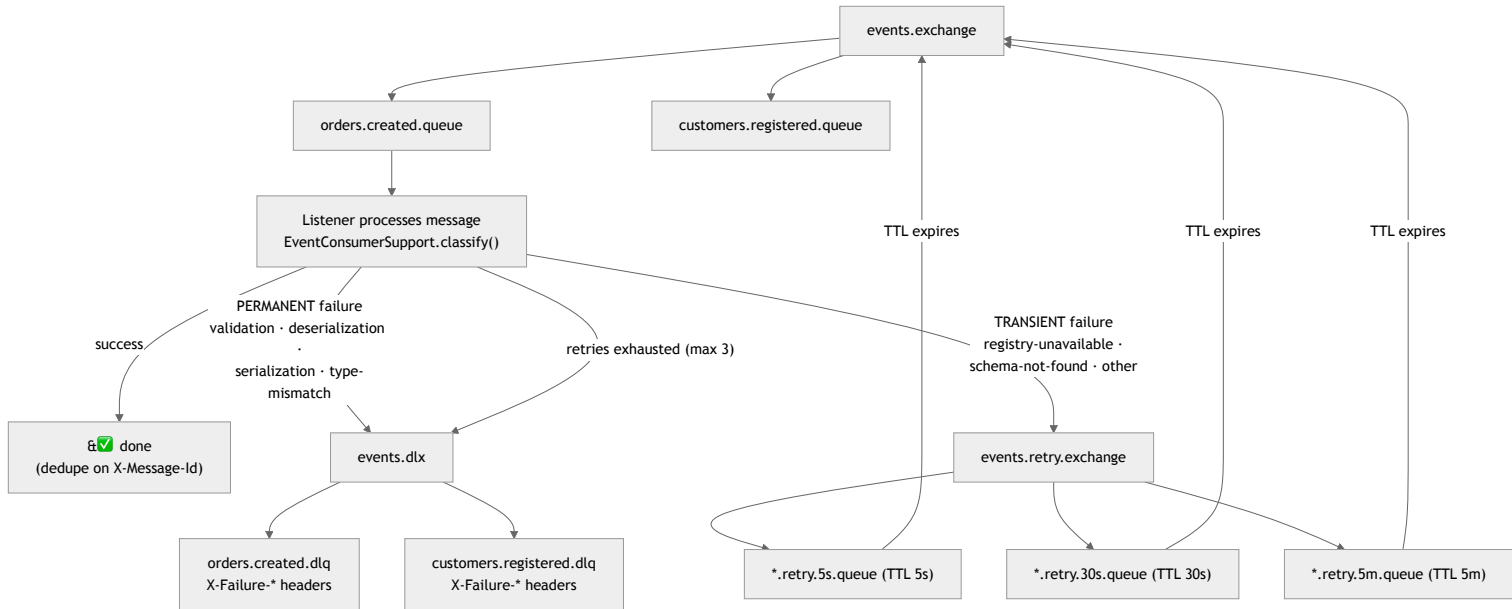
Runner topology & cost model

All three workflows run on a **self-hosted runner** (`[self-hosted, apicurio-local]` ,JDK 25 + `toolchains.xml`) that **shares the machine** with the standing registry at `localhost:8080` .

Aspect	With this setup
GitHub Actions minutes	None consumed — jobs run on your machine
Repo visibility	Must be private (fork-PR RCE risk on public)
Availability	If the machine is off, jobs queue — they don't fail
Speed	Fast — registry + <code>~/m2</code> already warm, no per-job startup
Scaling	Move runner + registry to an always-on VM, same label & workflows

Why self-hosted: the dry-run gate needs to reach a **live registry with real history**. Co-locating the runner and the registry makes that a `localhost` call — no secrets, no tunnels.

Failure model — the runtime safety net



Transient vs permanent — and forensics

The exception taxonomy **drives** the routing decision:

Exception	Class	Goes to
SchemaValidationException , DeserializationException , SerializationException , IncompatibleSchemaTypeException	Permanent	straight to DLQ
RegistryUnavailableException , SchemaNotFoundException , anything else	Transient	retry ladder (5s/30s/5m, max 3)

Every DLQ message carries `X-Failure-*` headers (reason, message, **stack trace truncated to 4 KB**, original routing key, timestamp, retry count). Consumer dedupes on `X-Message-Id` .

Live demo: `POST /api/orders/poison` publishes garbage bytes → `DeserializationException` (permanent) → lands on `orders.created.dlq` fully annotated. Inspect at `http://localhost:15672` .

Why Maven (and not Gradle — yet)

- Schema registration and the compat gate use the **official** `apicurio-registry-maven-plugin` —
there is **no trusted Gradle equivalent** for the dry-run/register goals.
- Maven is the POC build **by design**; a Gradle migration is **explicitly deferred post-POC**.
- Build conventions are **enforced**, not conventional:
 - Single source of versions in the parent POM (Spring Boot 4.0.x BOM); modules never re-declare.
 - Java 25 via `maven-toolchains-plugin` + `<release>25</release>` (bytecode 69).
 - `core ↗ contracts` enforced by `maven-enforcer-plugin` `bannedDependencies`.
- Test split is **load-bearing**: Surefire (`*Test.java` , fast, mock-based) vs Failsafe (`*IT.java` , Testcontainers, on `verify`).

Takeaways & adoption path

What this POC proves

1. One registry governs **multiple schema formats** (Protobuf *and* JSON Schema) uniformly.
2. Breaking changes are caught **on the PR**, as a red check — not in production.
3. The rule is **per-artifact** and must match the format (**Protobuf→BACKWARD, JSON→FORWARD**).
4. Registry contents are a **projection of `main`** ; resolution is **cached & outage-tolerant**.
5. Bad messages are **contained** (DLQ + retry ladder), not lost or silently dropped.

How a team rolls this out

- Stand up Apicurio (+ Postgres) → run **bootstrap** once per environment.
- Add the **compat-check** workflow + make it a **required status check** on `main`.
- Add **register** on merge. Pick each artifact's rule from how that format evolves.

Appendix

Command cheat sheet

Command cheat sheet

```
# --- Infra (Postgres, Apicurio + UI, RabbitMQ) ---
docker compose up                                # cold start → all-healthy

# --- Register schemas + attach rules (host cold-start) ---
./mvnw -pl order-contracts,customer-contracts apicurio-registry:register \
  -Dapicurio.registry.url=http://localhost:8080
#   OrderCreated (Protobuf)                → BACKWARD
#   CustomerRegistered (JSON Schema) → FORWARD

# --- Compat gate (what a PR runs; read-only dry-run) ---
./mvnw -pl order-contracts,customer-contracts verify -Pcompat-check

# --- Incompatible-change demos (MUST fail with INCOMPATIBLE) ---
./mvnw -pl order-contracts verify -Pincompatible-demo -Dapicurio.registry.url=http://localhost:8080
./mvnw -pl customer-contracts verify -Pincompatible-demo -Dapicurio.registry.url=http://localhost:8080

# --- Run services & exercise the DLQ ---
./mvnw -pl producer-service spring-boot:run      # :8081
./mvnw -pl consumer-service spring-boot:run      # :8082
curl -X POST http://localhost:8081/api/orders/poison # → orders.created.dlq

# --- Render this deck ---
npx -y @marp-team/marp-cli docs/presentation/apicurio-schema-governance-deck.md \
  --pdf --html --allow-local-files
```

Source material (in this repo)

This deck distills:

- `docs/short-tutorial.md` — compatibility-mode narrative + the three workflows
- `docs/CI-SCHEMA-TESTING-GUIDE.md` — change-type → outcome decision matrix + worked examples
- `docs/github_ci_steps.md` — workflow wiring, branch protection, cost model
- `docs/TECHNICAL-REPORT.md` — message lifecycle, resolution/caching, failure topology
- `.github/workflows/{schema-compat-check,schema-register,schema-governance-bootstrap}.yml`
- `CLAUDE.md` — architecture, wire format, exception taxonomy

Diagrams: editable Mermaid source in `docs/presentation/diagrams/*.mmd` ; rendered SVGs alongside.

Re-render with `npx -y @mermaid-js/mermaid-cli -i <file>.mmd -o <file>.svg -t neutral -b white`.